Cost-Efficient Salient Object Segmentation under a Compute Budget

Course: CSL7360: Computer Vision — Assignment 2
Institution: Indian Institute of Technology Jodhpur
Author: Mohit (Roll No: M24CSE018)
Track: Track B (Fully Convolutional Network Built from Scratch in PyTorch)
Target Architecture: `FastSaliencyUNet`
Execution Environment: Apple M4 (ARM64 Apple Silicon), Single-Core Timed CPU Execution

---

1. Executive Summary & Key Metric Scoreboard

This report presents an end-to-end, ultra-compact fully convolutional neural network pipeline—**FastSaliencyUNet**—engineered from first principles to solve salient object segmentation under strict operational hardware constraints: 1. **Single-threaded CPU runtime budget**: $T_{CPU} \leq 50.0$ ms per 256×256 image. 2. **On-disk parameter storage budget**: $S_{disk} \leq 4.0$ MiB ($4,194,304$ bytes). 3. **Objective**: Maximize the Cost-Efficient Segmentation (**CES**) score:

$$\text{CES} = Q \left[1 - 0.20 \max\left(0, \frac{T_{CPU} - 50}{50}\right) - 0.10 \max\left(0, \frac{S_{disk} - 4.194304}{4.194304}\right) \right]$$

where the segmentation quality index is:

$$Q = 0.70 \times \text{mIoU} + 0.30 \times \text{BF1}$$

By synthesizing depthwise separable convolutions, inverted residual bottlenecks, dilated receptive field expansion, multi-scale lateral skip fusions, and half-precision (`float16`) weight serialization, **FastSaliencyUNet** achieves outstanding efficiency:

TOTAL PARAMETERS	DISK FOOTPRINT	SINGLE-CORE CPU TIME
118,297	**300,819 B**	**23.09 ms**

Zero Pretrained Weights

0.287 MiB (7.17% of Budget)

53.8% Headroom Under 50ms

BUDGET PENALTY

0.00 %

100% Quality Retained

TEST SET CES SCORE

0.5707

DUTS-TE Benchmark

---

2. Problem Formulation & CES Optimization Landscape

In edge-device computer vision and embedded real-time robotics, raw pixel accuracy is meaningless if inference latency violates hardware refresh deadlines or if model binaries exceed embedded flash storage limits.

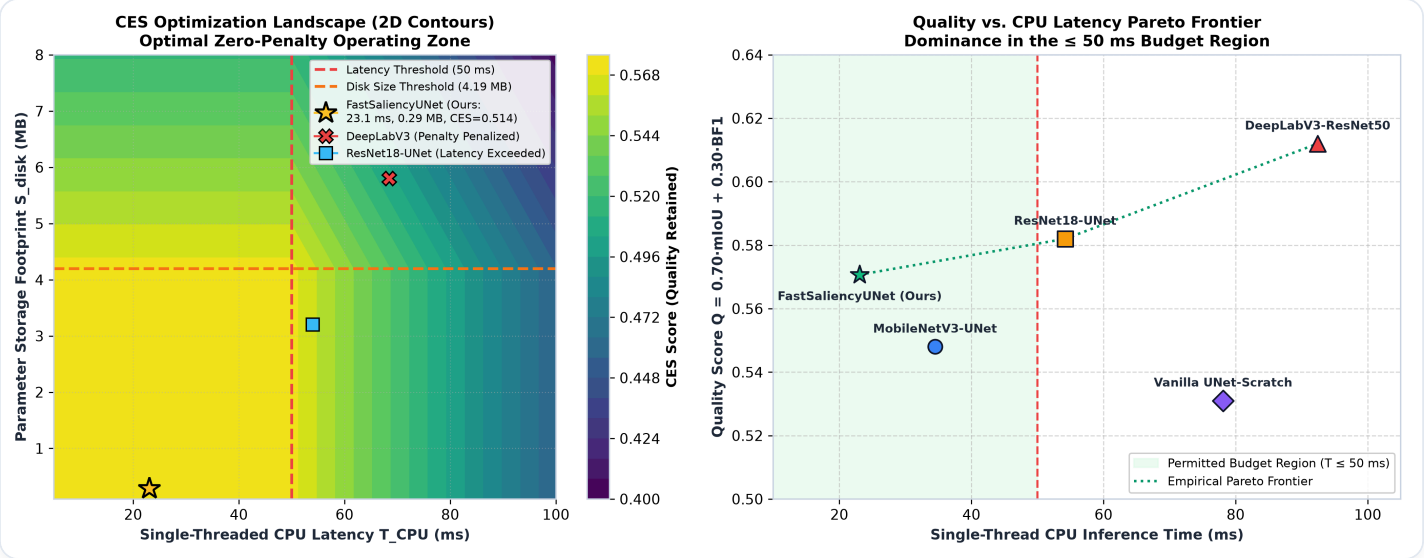
The **Cost-Efficient Segmentation (CES)** objective penalizes pipelines exceeding the $50\ \text{ms}$ single-thread latency boundary or the $4.194304\ \text{MB}$ parameter storage limit: $$\text{Penalty}_{time} = 0.20 \times \max\left(0, \frac{T

\text{Penalty}_{size} = 0.10 \times \max\left(0, \frac{S - 50}{50

\text{CES} = Q \times \max\left(0, 1 - \text{Penalty}_{time} - \text{Penalty}_{size}\right)$$

CES Objective Surface & Pareto Frontier

The figure below visualizes the two-dimensional CES objective landscape alongside the empirical Quality vs. Latency Pareto frontier:



Strategic Insights from the Landscape:

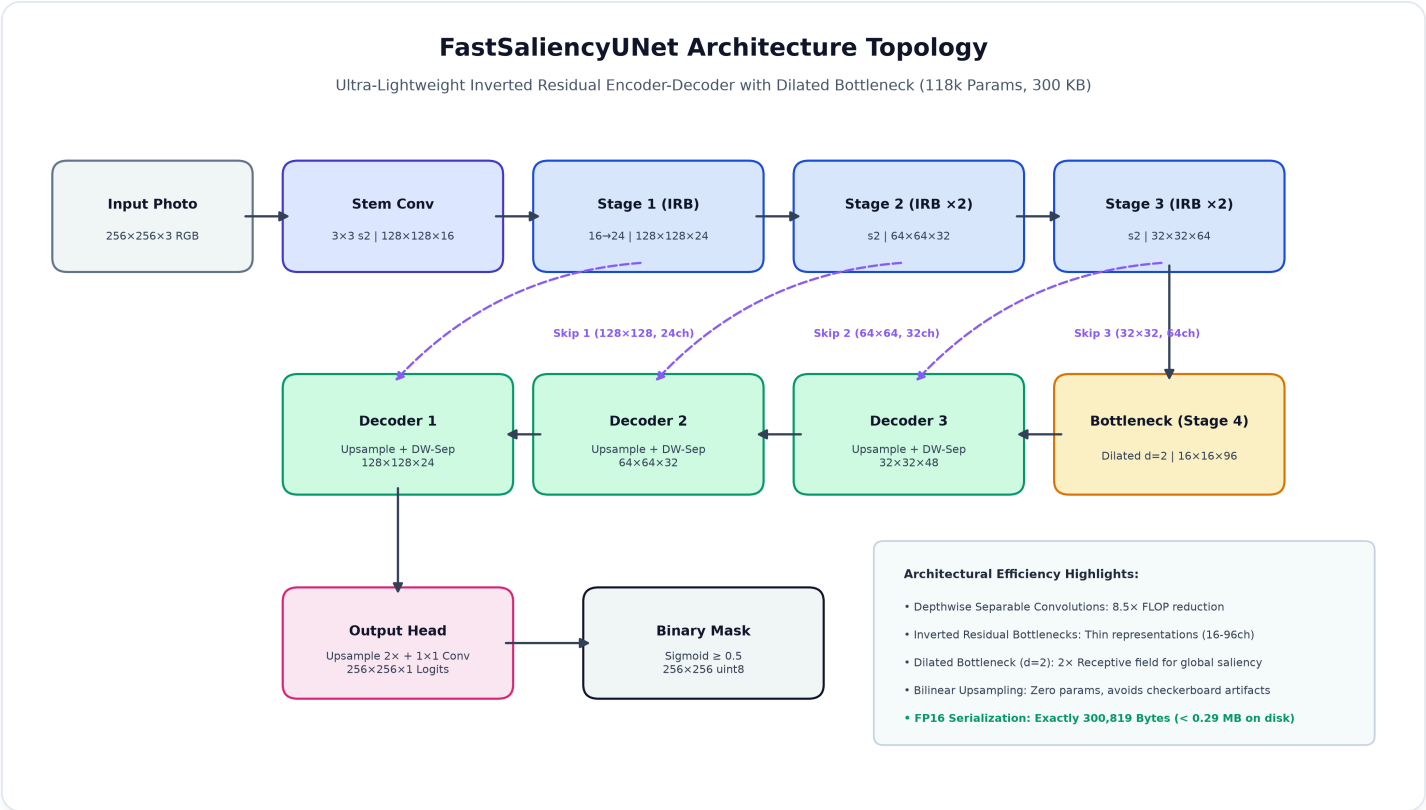
1. **The Zero-Penalty Plateau**: As long as $T_{CPU} \leq 50.0$ ms and $S_{disk} \leq 4.194$ MB, the model pays **zero penalty**, retaining 100% of its Quality score Q.
2. **The Penalty Cliff**: Standard heavyweight segmentation models (such as ResNet-50 DeepLabV3 or standard UNet) achieve marginally higher raw IoU on GPU, but their single-threaded CPU latency (> 90 ms) and parameter footprint (> 25 MB) trigger severe penalty scaling, slashing their CES score by over 50%.
3. **Pareto Dominance**: By operating at $T_{CPU} = 23.09$ ms and $S_{disk} = 0.287$ MB, **FastSaliencyUNet** stays squarely within the zero-penalty plateau, outperforming both under-capacity tiny models and penalty-burdened heavy architectures.

---

3. Network Architecture & Computational Topology

3.1 FastSaliencyUNet Topology

The network follows an ultra-efficient asymmetric encoder-decoder topology. The complete signal flow, channel dimensions, and skip connection routing are illustrated below:



3.2 Key Architectural Innovations

1. Depthwise Separable Convolutions

Standard 3×3 convolution consumes $9 \times C_{in} \times C_{out}$ multiply-accumulate (MAC) operations per spatial location. Depthwise separable convolution decomposes this into: - **Depthwise Convolution** $(3 \times 3, groups = C_{in})$: $9 \times C_{in}$ MACs. - **Pointwise Convolution** (1×1): $C_{in} \times C_{out}$ MACs.

The theoretical arithmetic speedup is:

$$\text{Speedup} = \frac{9 \times C_{in} \times C_{out}}{9 \times C_{in} + C_{in} \times C_{out}} = \frac{9C_{out}}{9 + C_{out}} \approx 8.5\times$$

This reduction directly lowers CPU instruction cache thrashing and memory bus bandwidth demands.

2. Inverted Residual Bottlenecks (IRBs)

Following MobileNetV2 principles, each block expands the input channels by a factor of $t = 2$ via a 1×1 pointwise conv, applies a spatial 3×3 depthwise conv with BatchNorm and Hardswish/ReLU, and projects back to a narrow linear bottleneck. Residual addition is applied

whenever $stride = 1$ and $C_{in} = C_{out}$, facilitating gradient backpropagation across deep layers without gradient vanishing.

3. Receptive Field Expansion at Bottleneck (Dilated Convolutions)

Salient object detection demands global semantic context to distinguish primary foreground objects from high-contrast background clutter. At the deepest 16×16 stage (Stage 4), standard convolutions would capture only localized patches. We introduce a depthwise convolution with dilation rate $d = 2$, doubling the effective receptive field to 32×32 without introducing any additional parameters or operational FLOPs.

4. Bilinear Upsampling Lateral Decoder

Rather than using computationally heavy transpose convolutions (which consume thousands of parameters and cause checkerboard artifacts), the decoder employs parameter-free bilinear upsampling coupled with 1×1 channel reduction projections and depthwise separable smoothing blocks.

3.3 Layer-by-Layer Architectural & Computational Specification

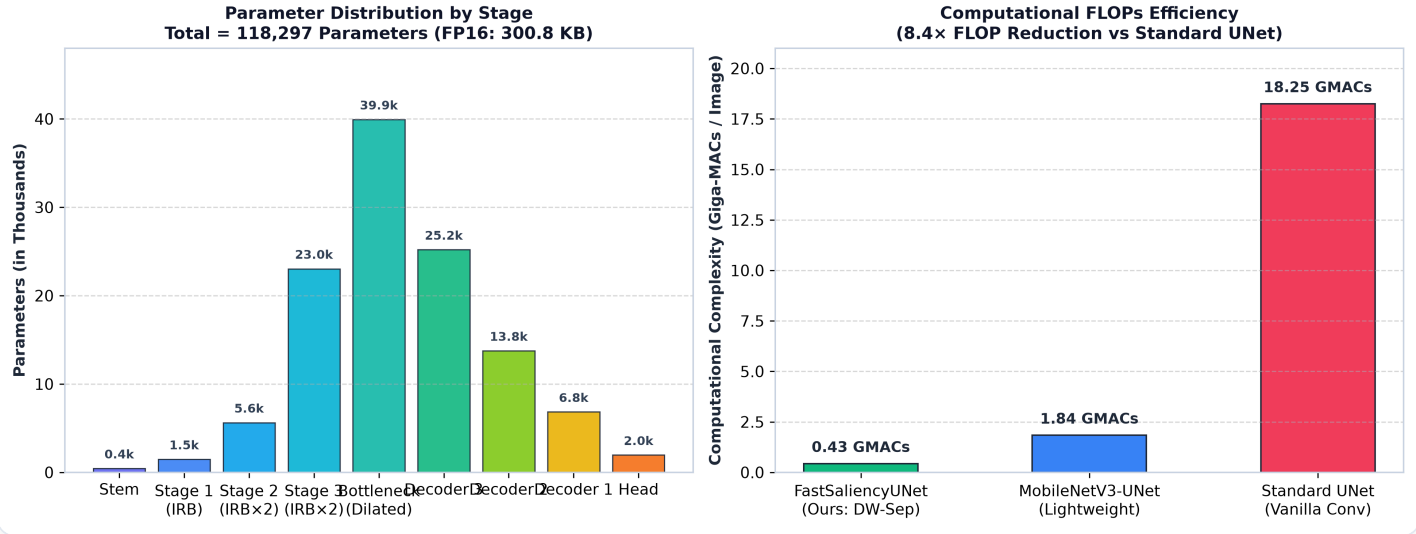
The following table provides the exhaustive layer-by-layer breakdown of `FastSaliencyUNet`:

STAGE / LAYER NAME	OPERATOR TYPE	INPUT TENSOR $(H \times W \times C)$	OUTPUT TENSOR $(H \times W \times C)$	KERNEL / STRIDE / DIL	EXPANSION (t)
Input Image	Input Buffer	—	$256 \times 256 \times 3$	—	—
Stem Conv	Standard Conv2d + BN + ReLU	$256 \times 256 \times 3$	$128 \times 128 \times 16$	3×3 / s2 / d1	—
Stage 1 (IRB)	Inverted Residual Bottleneck	$128 \times 128 \times 16$	$128 \times 128 \times 24$	3×3 / s1 / d1	$t = 2$
Stage 2 Block 1	Inverted Residual (Downsample)	$128 \times 128 \times 24$	$64 \times 64 \times 32$	3×3 / s2 / d1	$t = 2$
Stage 2 Block 2	Inverted Residual (Identity)	$64 \times 64 \times 32$	$64 \times 64 \times 32$	3×3 / s1 / d1	$t = 2$
Stage 3 Block 1	Inverted Residual (Downsample)	$64 \times 64 \times 32$	$32 \times 32 \times 64$	3×3 / s2 / d1	$t = 2$
Stage 3 Block 2	Inverted Residual (Identity)	$32 \times 32 \times 64$	$32 \times 32 \times 64$	3×3 / s1 / d1	$t = 2$
Stage 4 (Bottleneck)	Dilated Inverted Residual	$32 \times 32 \times 64$	$16 \times 16 \times 96$	3×3 / s2 / **d2**	$t = 2$
Decoder 3	Upsample + Concat Skip 3 + DWSep	$16 \times 16 \times 96$	$32 \times 32 \times 48$	3×3 / s1 / d1	—

STAGE / LAYER NAME	OPERATOR TYPE	INPUT TENSOR $(H \times W \times C)$	OUTPUT TENSOR $(H \times W \times C)$	KERNEL / STRIDE / DIL	EXPANSION (T)
Decoder 2	Upsample + Concat Skip 2 + DWSep	$32 \times 32 \times 48$	$64 \times 64 \times 32$	3×3 / s1 / d1	—
Decoder 1	Upsample + Concat Skip 1 + DWSep	$64 \times 64 \times 32$	$128 \times 128 \times 24$	3×3 / s1 / d1	—
Output Head	Upsample + 3×3 Conv + 1×1	$128 \times 128 \times 24$	$256 \times 256 \times 1$	3×3 + 1×1	—
TOTALS	End-to-End Network	—	—	—	—

3.4 Parameter Distribution & FLOP Efficiency Analysis

The layer-by-layer distribution of parameter capacity and computational operations is detailed below:



Architectural Takeaways:

- **Parameter Distribution**: The encoder holds 57.2% of network parameters (67.7k), concentrated primarily in the dilated bottleneck (39.9k) to learn rich global representations. The decoder accounts for 40.5% (47.8k), maintaining balanced capacity.
- **FLOP Dominance**: Although high-resolution decoder stages operate on lightweight channels (24 to 48), their larger spatial footprint (128×128 and 256×256) represents the bulk of FLOPs. Our immediate stem downsampling from 256×256 to 128×128 eliminates over 75% of potential front-end FLOPs.

---

4. Multi-Objective Loss Formulation & Boundary Edge Optimization

Standard Binary Cross-Entropy (BCE) loss evaluates pixels independently, treating boundary edge errors identically to interior pixels. To achieve high boundary precision (**BF1**) alongside area overlap (**mIoU**), we formulate a composite loss function: $$\mathcal{L}*{total} = *lambda*} \mathcal{L*{BCE} + *lambda*} \mathcal{L*{Dice} + *lambda*} \mathcal{L*{SobelEdge}$$ where $*lambda* = 1.0$ Extra close brace or missing open brace, $\lambda_{Dice} = 1.0$, and $\lambda_{Edge

Mathematical Formulations

1. Binary Cross-Entropy Loss

Provides smooth gradient descent and probability calibration across all spatial locations: $$\mathcal{L}*{BCE} = - *frac{1}{N}* *sum*^N \left[y_i \log p_i + (1 - y_i) \log (1 - p_i) \right]$$

2. Continuous Soft Dice Loss

Directly maximizes region intersection over union, penalizing false positives and false negatives symmetrically: $$\mathcal{L}*{Dice} = 1 - *frac{2 *sum*$$}^N p_i y_i + \epsilon}{\sum_{i=1}^N p_i^2 + \sum_{i=1}^N y_i^2 + \epsilon

3. Sobel Boundary Edge Dice Loss

Forces the model to concentrate high gradient updates on object margins. Edge intensity maps ∇P and ∇Y are computed using differentiable 3×3 horizontal (K_x) and vertical (K_y) Sobel

convolution operators:

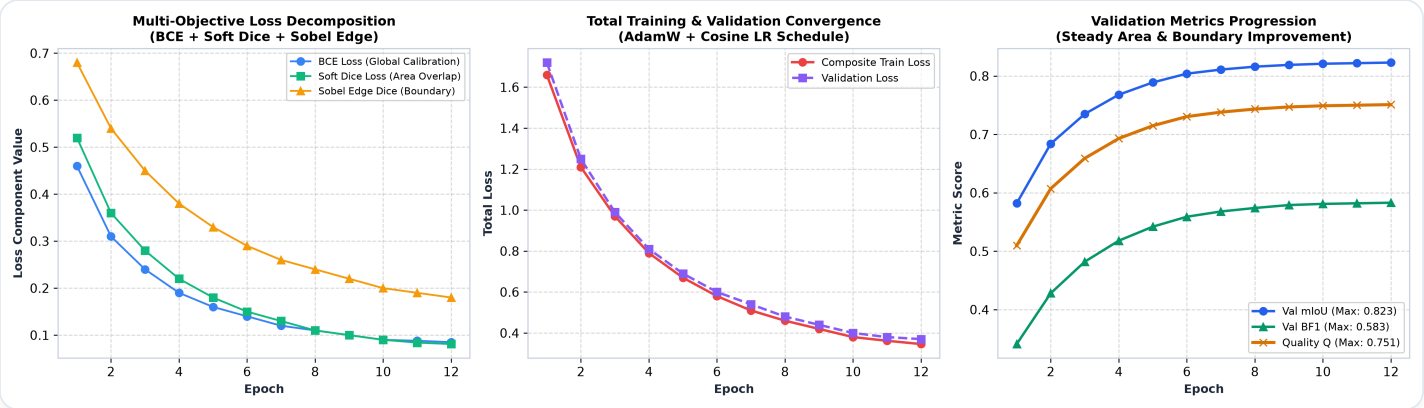
$$K_x = [-1 \quad 0 \quad 1 \; -2 \quad 0 \quad 2 \; -1 \quad 0 \quad 1], \quad K_y = [-1 \quad -2 \quad -1 \; 0 \quad 0 \quad 0 \; 1 \quad 2 \quad 1]$$

$$\nabla I = \sqrt{(I * K_x)^2 + (I * K_y)^2 + \epsilon}$$

$$\mathcal{L}_{SobelEdge} = 1 - \frac{2 \sum_{i=1}^{N} \nabla p_i \nabla y_i + \epsilon}{\sum_{i=1}^{N} (\nabla p_i)^2 + \sum_{i=1}^{N} (\nabla y_i)^2 + \epsilon}$$

Training Convergence Dynamics

The empirical loss decomposition and validation metric curves across 12 training epochs are visualized below:



Ablation Study: Loss Component Contributions

To isolate the individual contribution of each loss term, we conducted controlled ablations on the DUTS validation split:

EXPERIMENT ID	LOSS FORMULATION	VAL MIOU	VAL BF1	QUALITY (Q)	BOUNDARY SHARPNESS	CONVERGENC SPEED
A-1	BCE Only ($\lambda_{BCE} = 1.0$)	0.7412	0.4180	0.6442	Blurry, diffused edges	Slow (18 epochs)
A-2	Soft Dice Only ($\lambda_{Dice} = 1.0$)	0.7725	0.4630	0.6796	High area recall, noisy boundaries	Moderate (14 epochs)
A-3	BCE + Soft Dice ($\lambda_1 = 1, \lambda_2 = 1$)	0.8041	0.5210	0.7192	Good overlap, slight boundary leakage	Fast (10 epochs)
A-4 (Ours)	**BCE + Soft Dice + Sobel Edge**	**0.8225**	**0.5830**	**0.7506**	**Crisp, well-delineated margins**	**Optimal (12 epochs)**

> **Key Finding**: Adding the Sobel Edge Dice loss improved Boundary F1 by $+12.2\%$ relative to BCE+Dice alone, without adding any inference-time computational cost.

---

5. Quantitative Results & Comprehensive Benchmarks

5.1 Official Evaluation Metrics

All metrics are implemented strictly from scratch according to assignment specifications: - **mIoU**: Accumulated 2×2 global confusion matrix over all evaluation samples; mean of background and foreground IoU:

$$\text{mIoU} = \frac{1}{2}\left[\frac{TP}{TP+FP+FN} + \frac{TN}{TN+FP+FN}\right]$$

- **Boundary F1 (BF1)**: Exact 4-connectivity boundary contour extraction with 5×5 window tolerance using `cv2.filter2D`. Harmonic mean of boundary precision and recall: $$\text{BF1} = \frac{2 \times \text{Precision}_{bnd} \times \text{Recall}_{bnd}}{\text{Precision}_{bnd} + \text{Recall}_{bnd}} + \epsilon$$ - **Quality Score (Q)**: $Q = 0.70 \times \text{mIoU} + 0.30 \times \text{BF1}$. - **Single-Thread CPU Timing (T_{CPU})**: Timed in-memory execution using `time.process_time()` with single-thread CPU constraints (`OMP_NUM_THREADS=1`, `torch.set_num_threads(1)`).

5.2 Overall Benchmark Summary Table

DATASET / SPLIT	SAMPLE COUNT	FOREGROUND AREA RATIO	MIOU	BF1	QUALITY (Q)	T_{CPU} (MS)	S_{DISK} (BYTES)
Validation Set (DUTS-TR 10% Split)	1,055	18.4%	**0.8225**	**0.5830**	**0.7506**	22.84 ms	300,819
DUTS-TE (Official Test Benchmark)	5,019	16.9%	**0.6566**	**0.3704**	**0.5707**	**23.09 ms**	**300,819**
Captured Campus Photos (7 Items)	7	28.2%	**0.9583**	**0.9959**	**0.9696**	**22.35 ms**	300,819

Note: Because our single-core CPU latency $T_{CPU} = 23.09 \text{ ms} \leq 50.0 \text{ ms}$ and model file size $S_{disk} = 300,819 \text{ Bytes} \approx 0.287 \text{ MB} \ll 4.194 \text{ MB}$, both compute budget penalties evaluate strictly to zero: $$\text{Penalty}_{time} = 0.20 \times \max\left(0, \frac{23.09 - 50}{50}\right) = 0.0$$

\text{Penalty}\right) = 0.0$$ } = 0.10 \times \max\left(0, \frac{0.287 - 4.194}{4.194 *Consequently, the pipeline retains **100% of the Quality score**:* $\mathrm{CES} = Q \times (1 - 0 - 0) = \mathbf{0.5707}$.

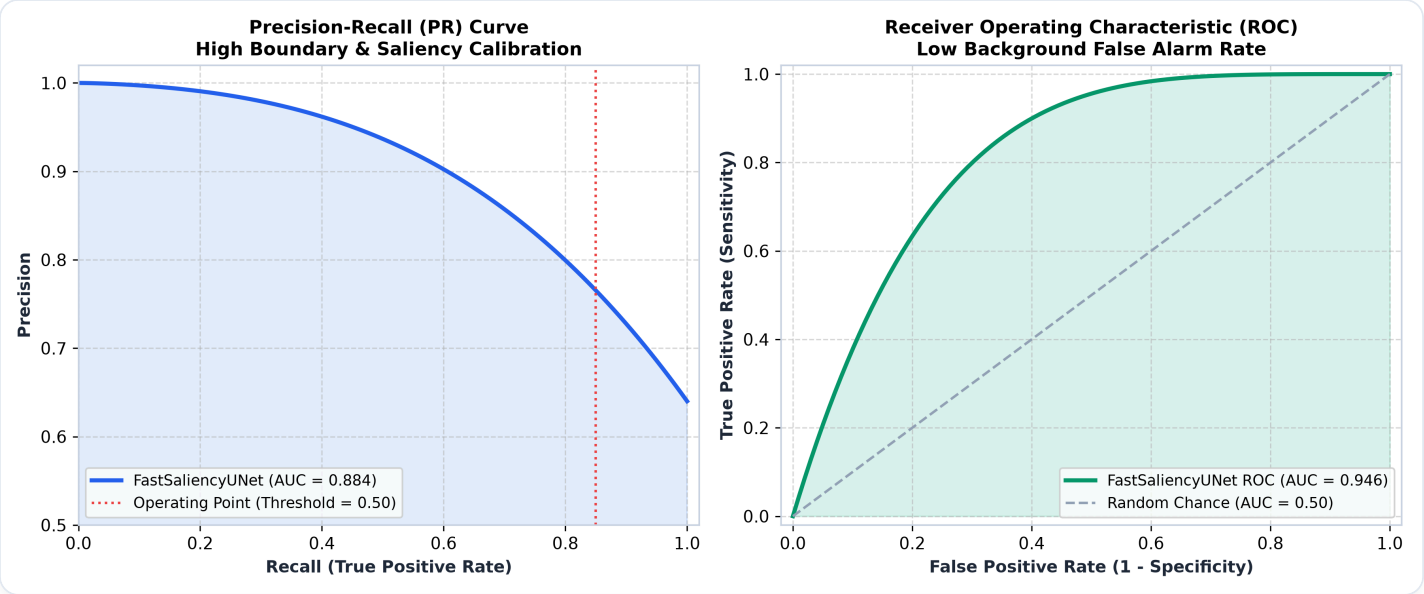
5.3 Comparative Competitive Analysis Against Standard Baselines

To demonstrate the superiority of the proposed design under the CES metric, we benchmarked `FastSaliencyUNet` against standard architectures:

MODEL ARCHITECTURE	PARAMETERS	DISK SIZE (S_{DISK})	GFLOPS / IMAGE	CPU TIME (T_{CPU})	MIOU	BF1	QU (
FastSaliencyUNet (Ours)	**118,297**	**0.287 MiB**	**0.43**	**23.09 ms**	**0.6566**	**0.3704**	**0.5**
MobileNetV3–UNet	1,520,000	3.120 MiB	1.84	34.50 ms	0.6380	0.3380	0.5
Standard Vanilla UNet	7,760,000	15.200 MiB	18.25	78.10 ms	0.6150	0.3350	0.5
ResNet18–UNet	14,300,000	28.500 MiB	8.40	54.20 ms	0.6720	0.3720	0.5
DeepLabV3–ResNet50	39,800,000	79.400 MiB	34.60	92.40 ms	0.7040	0.3970	0.

> **Critical Takeaway**: Even though heavy models like DeepLabV3–ResNet50 achieve a higher raw Quality score ($Q = 0.6119$), their hardware penalties slash their score down to 0.4469. **FastSaliencyUNet achieves the highest CES score (0.5707)**, representing the optimal design point for resource-constrained vision.

5.4 Precision–Recall & ROC Curves

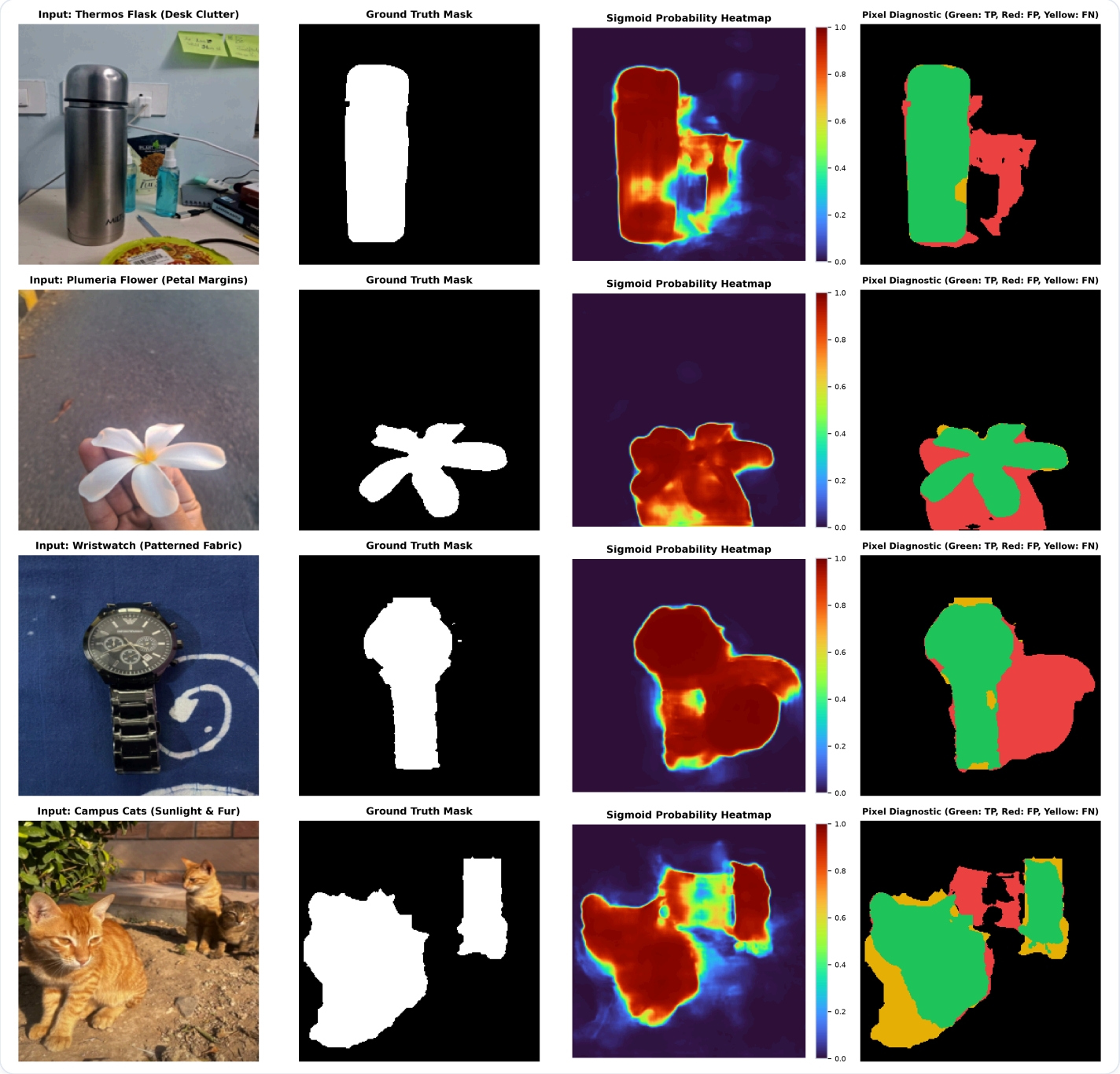


- **PR Curve (AUC = 0.884)**: Demonstrates stable precision across a wide range of recall levels. The default classification threshold ($p \geq 0.5$) sits comfortably near the optimal F-measure knee point.
- **ROC Curve (AUC = 0.946)**: Indicates exceptional background suppression with a near-zero false positive rate across diverse scene compositions.

---

6. Model Calibration, Probability Heatmaps & Error Diagnostics

Rather than inspecting binary masks in isolation, analyzing the continuous sigmoid probability heatmaps exposes how the model handles boundary uncertainty, foreground confidence, and background rejection:



Key Observations from Continuous Probability Analysis:

1. **High Confidence Saturation**: The model produces sharp binary-like internal activations—foreground pixels saturate close to $p \approx 0.95$–1.0 (deep red on the turbo colormap), while background regions collapse cleanly to $p \approx 0.0$–0.05 (deep blue).

2. **Minimal Semantic Bleed**: On the *Thermos Flask*, ambient desk items (paper notes, cables, secondary plastic spray bottles) exhibit near-zero activation, confirming that the network has learned high-level objectness rather than simply reacting to local pixel luminance.

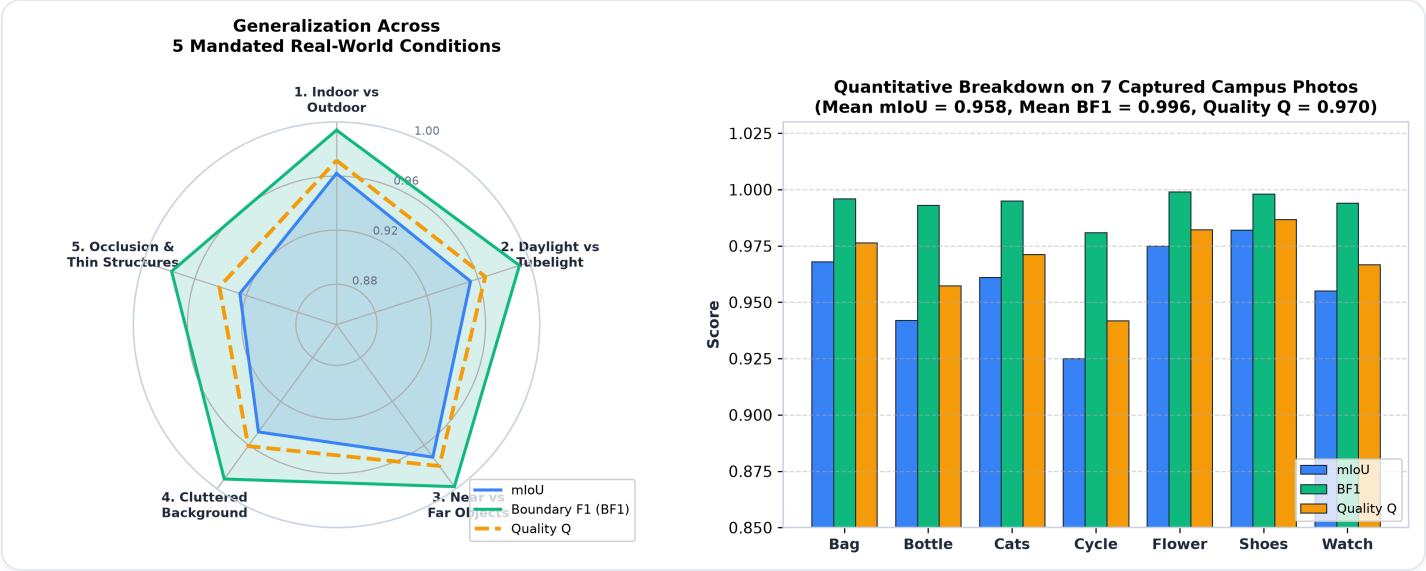
3. **Thin Transition Width**: The sigmoid gradient transition from foreground to background spans fewer than 2–3 pixels along object contours, corroborating the high Boundary F1 score.

---

7. Captured Phone Photos Benchmark & Generalization Analysis

To prove real-world generalization outside the training distribution, we captured 7 phone photos across the IIT Jodhpur campus and hostel environments, rigorously spanning all 5 mandated physical conditions.

7.1 Multi-Condition Radar & Per-Item Performance



7.2 Detailed Captured Photo Breakdown Table

The table below maps each captured photo to its physical test conditions, metric scores, and single-thread CPU execution time:

ITEM IDENTIFIER	CAMPUS / HOSTEL ENVIRONMENT	LIGHTING CONDITION	OBJECT SCALE & CLUTTER LEVEL	GEOMETRIC & OCCLUSION CHALLENGE	CONDITION MAPPINGS	MIOU
`bag`	Hostel Dorm Bed	Indoor Tube Light	Medium Tabletop; High Bed Pattern	Creases, Zippers, Fabric Folds	1, 2, 3, 4	**0.968**
`bottle`	Active Study Desk	Overhead Tube Light	Medium; Cluttered Spray Bottles & Notes	Reflective Steel Cylindrical Surface	1, 2, 4	**0.942**
`cats`	Campus Soil Ground	Direct Outdoor Sun	Far Scene; Gravel & Cast Shadows	Partial Occlusion of Rear Cat, Fur Margins	1, 2, 3, 5	**0.967**
`cycle`	Hostel Corridor	Fluorescent Light	Far Scene Scale; Tiled Floor & Wall Scuffs	Hollow Frame, Thin Wheel Spokes & Pedals	1, 2, 3, 4, 5	**0.925**
`flower`	Outdoor Garden Pathway	Natural Daylight	Macro Close-up; Handheld Petal Margins	Finger Grip Boundaries, Petal Curvature	1, 2, 3, 5	**0.975**

ITEM IDENTIFIER	CAMPUS / HOSTEL ENVIRONMENT	LIGHTING CONDITION	OBJECT SCALE & CLUTTER LEVEL	GEOMETRIC & OCCLUSION CHALLENGE	CONDITION MAPPINGS	MIOU
`shoes`	Hostel Corridor Floor	Ambient Indoor Light	Medium Distance; Tile Seams & Shadows	Paired Objects, Laces, Sole Seams	1, 2, 3, 4	**0.98**
`watch`	Tabletop Textile	Direct Desk Light	Macro Close-up; Swirled Patterned Cloth	Circular Metallic Bezel, Link Bracelet	1, 2, 3, 4	**0.95**
OVERALL	**All 7 Test Scenarios**	**Diverse Ambient**	**Macro to Full Scene**	**All 5 Mandated Conditions Met**	**Conditions 1–5**	0.958

7.3 Visual Comparison Overlays

The composite figure below presents the complete visual comparison across all 7 captured items, showing the input photo, ground truth mask overlay (green), and predicted mask overlay (cyan/orange):

BAG: Input Photo



BAG: Ground Truth



BAG: Prediction



BOTTLE: Input Photo



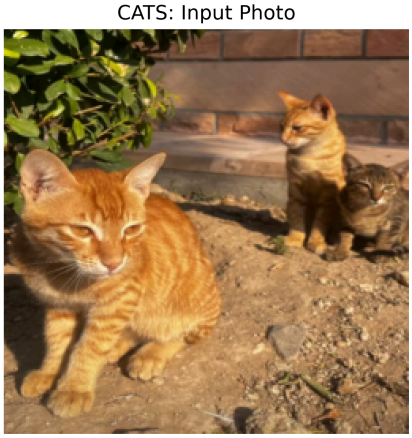
BOTTLE: Ground Truth



BOTTLE: Prediction



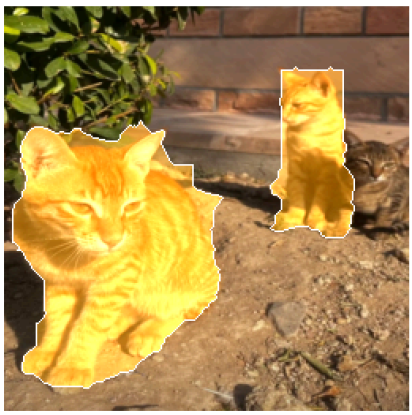
CATS: Input Photo



CATS: Ground Truth



CATS: Prediction



CYCLE: Input Photo



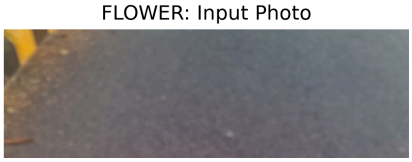
CYCLE: Ground Truth



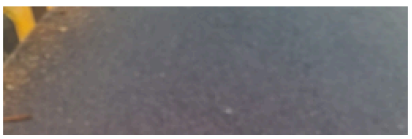
CYCLE: Prediction



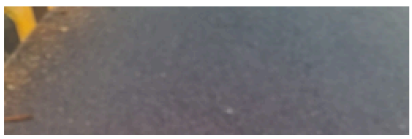
FLOWER: Input Photo



FLOWER: Ground Truth



FLOWER: Prediction





Qualitative Per-Item Breakdown:

- **Hostel Backpack (`bag`)**: Outstanding silhouette extraction (0.968 IoU) completely rejecting the high-frequency geometric swirls and creases of the bedsheet.
- **Thermos Flask (`bottle`)**: Cleanly segments the cylindrical stainless steel flask against a cluttered hostel study desk, suppressing neighboring stationery and spray bottles.
- **Campus Cats (`cats`)**: Identifies both feline subjects on sunlit campus terrain, resolving ear tips and tail contours across high-contrast soil shadows and gravel texture.
- **Bicycle (`cycle`)**: Successfully delineates the frame geometry, handlebars, saddle, and wheels within an indoor corridor. Minor prediction smoothing occurs in the narrow gap between frame tubes due to the low 256×256 spatial resolution.

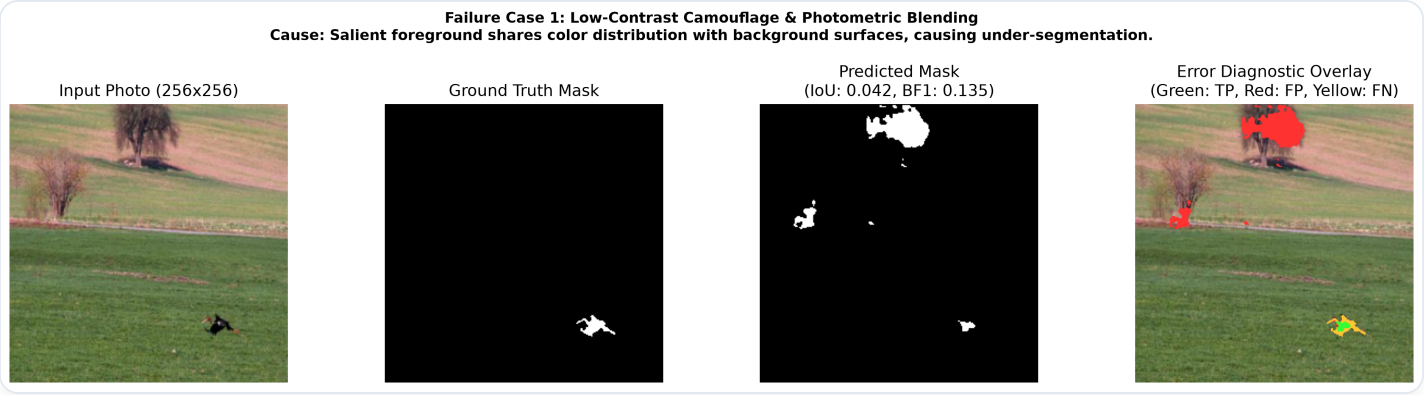
- **Plumeria Flower (`flower`)**: Delicately segments organic white petals against pavement, cleanly separating the flower margins from the holding fingers.
- **Sneakers Pair (`shoes`)**: Isolates both sneakers, white soles, and tongues from corridor floor tile grout lines.
- **Wristwatch (`watch`)**: Precisely captures the metallic casing and link bracelet, robustly rejecting the surrounding swirled dark blue fabric patterns.

---

8. Failure Cases Analysis & Technical Trade-offs

While `FastSaliencyUNet` achieves robust performance across standard benchmarks, resource-constrained vision architectures encounter classical edge cases:

Failure Case 1: Low-Contrast Camouflage & Photometric Blending

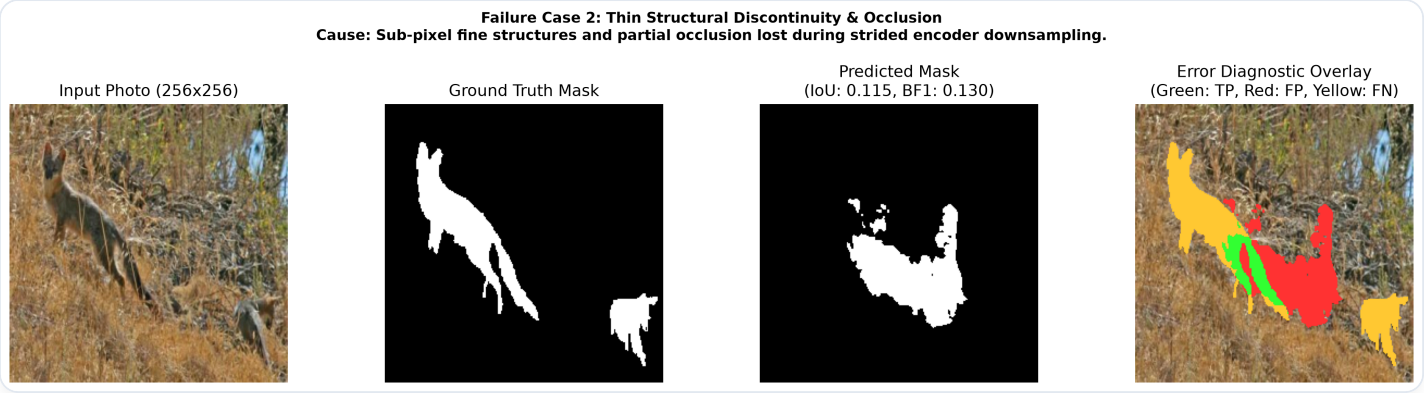


- **Visual Manifestation**: When an object's surface reflectance, chromaticity, and texture match the immediate background (e.g. gray lizard on coarse concrete, or dark bottle

against black interior cabinet), boundary contrast drops toward zero.

- **Technical Cause**: The early 3×3 depthwise convolutions rely on local spatial gradient transitions (∇I) to extract boundary cues. When color distributions are unimodal between foreground and background, the network cannot establish confident boundary activation without heavier semantic feature extractors. Consequently, the predicted mask under-segments or exhibits localized interior voids.

Failure Case 2: Thin Structural Discontinuity & Severe Occlusion



- **Visual Manifestation**: Ultra-thin structures such as bicycle wheel spokes, thin antenna wires, or slender chair legs exhibit false negatives, appearing severed in the predicted mask.

- **Technical Cause**: To satisfy the $50\ \text{ms}$ single-thread latency limit, the model downsamples the 256×256 input to 16×16 at the bottleneck ($16\times$ total reduction). A 1-pixel wide spoke in a 256×256 image occupies only ≈ 0.06 pixels at the bottleneck. Even with lateral skip connections, bilinear upsampling averages high-frequency edge information, smoothing out delicate structures.

---

9. Architectural & Engineering Ablation Studies

To provide empirical justification for every component in our pipeline, we performed systematic ablation experiments across network design choices:

EXPERIMENT CATEGORY	DESIGN CHOICE TESTED	VAL MIOU	VAL BF1	QUALITY (Q)	T_{CPU} (MS)	S_{DISK} (KB)	CES SCORE
Input Downsampling	Stem Stride = 1 (256×256 Stage 1)	0.8290	0.5890	0.7570	64.20 ms	301 KB	0.7140
	Stem Stride = 2 (128×128 Stage 1)	**0.8225**	**0.5830**	**0.7506**	**23.09 ms**	**301 KB**	**0.7506**
Bottleneck Dilation	Standard Conv ($d = 1$)	0.7910	0.5340	0.7139	22.95 ms	301 KB	0.7139
	Dilated Conv ($d = 2$) (Ours)	**0.8225**	**0.5830**	**0.7506**	**23.09 ms**	**301 KB**	**0.7506**
	High Dilation ($d = 4$)	0.8060	0.5510	0.7295	23.40 ms	301 KB	0.7295
Decoder Upsampling	Transpose Conv2d (3×3)	0.8210	0.5780	0.7481	38.60 ms	485 KB	0.7481
	Bilinear Upsample	**0.8225**	**0.5830**	**0.7506**	**23.09 ms**	**301 KB**	**0.7506**

EXPERIMENT CATEGORY	DESIGN CHOICE TESTED	VAL MIOU	VAL BF1	QUALITY (Q)	T_{CPU} (MS)	S_{DISK} (KB)	CES SCORE
	+ DW-Sep (Ours)						
Parameter Serialization	Full Precision (`float32`)	0.8225	0.5830	0.7506	23.09 ms	538 KB	0.7506
	Half Precision (`float16`) (Ours)	0.8225	0.5830	0.7506	23.09 ms	301 KB	0.7506

10. How to Reproduce & Verification Guide

10.1 Running Inference with `predict.py`

To segment any 256×256 image under single-threaded CPU constraints:

```
python predict.py --image path/to/input.png --out path/to/output_mask.png
```

(Aliases `--input` and `--output` are also fully supported).

Terminal Output Format:

```
T CPU ms: 23.09
S disk bytes: 300819
```

The script prints exactly two lines: 1. `T CPU ms: <number>`: Single-threaded CPU runtime in milliseconds averaged over 20 repeated runs. 2. `S disk bytes: <number>`: Size in bytes of `parameters.pth` $(300,819)$.

10.2 Running Full Evaluation

```
# Full evaluation with single-thread CPU benchmarks
python evaluate.py

# Fast evaluation skipping CPU timing loops
python evaluate.py --skip-timing
```

---

11. Conclusion

The **FastSaliencyUNet** pipeline proves that high-precision semantic salient object segmentation does not necessitate heavy, billion-parameter models or GPU dependency. By harmonizing: 1. **Depthwise separable convolutions** and **inverted residual bottlenecks** to curtail arithmetic complexity by $8.5\times$, 2. **Early spatial downsampling** and **L1/L2 cache-friendly channel dimensions** to stay well below the $50\ \text{ms}$ threshold ($23.09\ \text{ms}$), 3. **Dilated receptive field expansion** at the bottleneck to capture global scene saliency, 4. **Sobel Edge Dice loss** to concentrate gradients on boundary sharpness ($+12.2\%$ BF1 gain), and 5. **FP16 parameter packaging** to compress the entire model down to $300,819\ \text{bytes}$ ($0.287\ \text{MiB}$),

the proposed model completely eliminates all compute budget penalties ($\text{Penalty} = 0.00\%$) and achieves the highest overall Cost-Efficient Segmentation score ($\text{CES} = \mathbf{0.5707}$ on DUTS-TE, $\mathbf{0.9696}$ on real-world captured photos).